

Musiteca

Prerequisitos

Fichero de datos *metal-musics-dataset.csv* (descargar [aquí](#)) convenientemente montado en el contenedor *ksqldb-server*



Info

Este ejemplo muestra los fundamentos sobre conectores y comandos *ksql* en *Kafka*. En particular, se crea un conector *source* de la clase [FilePulse](#). Esto es posible porque el correspondiente plugin ha sido instalado previamente en el contenedor *ksqldb-server*. Para ver la lista de plugins instalados se puede invocar la siguiente orden (ver [comandos útiles](#)) en el terminal de comandos:

```
docker exec -it ksqldb-server curl -s localhost:8083/connector-plugins|jq '.  
[].class'
```

Para más información relativa al uso de curl con *API REST* ver [este enlace](#)

API REST: Transformaciones embebidas en la carga

Si se dispusiera de un cluster *Kafka* completo sería posible gestionar los conectores invocando el **API Rest** del conector. A continuación se muestra la orden suponiendo que el contenedor exporta y escucha en el puerto 8083 de la máquina local

```
-- Crear conector para cargar datos desde csv en un repo denominado Datasets
-- Incluye parsing cabeceras, conversión de tipo y adición del filename
-- Crea topic connect-file-pulse
--
curl -X POST http://localhost:8083/connectors -H "Content-Type:
application/json" -d '{
  "name": "connect-file-pulse-csv",
  "config": {

"connector.class":"io.streamthoughts.kafka.connect.filepulse.source.FilePulseS
ourceConnector",
  "filters": "ParseDelimitedRow,ReleaseToInt,FileName",
  "filters.ParseDelimitedRow.extractColumnName": "headers",
  "filters.ParseDelimitedRow.trimColumn": "true",
  "filters.ParseDelimitedRow.type":
"io.streamthoughts.kafka.connect.filepulse.filter.DelimitedRowFilter",
  "filters.ReleaseToInt.type":
```

```

"io.streamthoughts.kafka.connect.filepulse.filter.AppendFilter",
  "filters.ReleaseToInt.field": "$value.release",
  "filters.ReleaseToInt.value": "{{ converts($value.release, '\\'INTEGER\\')
}}",
  "filters.ReleaseToInt.overwrite": "true",
  "filters.FileName.type":
"io.streamthoughts.kafka.connect.filepulse.filter.AppendFilter",
  "filters.FileName.field": "$.filename",
  "filters.FileName.value": "$metadata.name",
  "fs.scanner.class":
"io.streamthoughts.kafka.connect.filepulse.scanner.local.LocalFSDirectoryWalke
r",
  "fs.cleanup.policy.class":
"io.streamthoughts.kafka.connect.filepulse.clean.LogCleanupPolicy",

"fs.scan.filters":"io.streamthoughts.kafka.connect.filepulse.scanner.local.fil
ter.RegexFileListFilter",
  "file.filter.regex.pattern": ".*\\.csv$",
  "fs.scan.directory.path": "/Datasets/",
  "fs.scan.interval.ms": "10000",
  "internal.kafka.reporter.bootstrap.servers": "broker:29092",
  "internal.kafka.reporter.topic": "connect-file-pulse-status",
  "offset.strategy": "name",
  "skip.headers": "1",
  "task.reader.class":
"io.streamthoughts.kafka.connect.filepulse.reader.RowFileInputReader",
  "topic": "connect-file-pulse",
  "tasks.max": 1
}
}'

```

KSQl: Base de Datos Kafka

En la infraestructura que se suministra la manipulación de los conectores se realiza invocando **órdenes *ksql***. En particular, la orden análoga a la llamada *REST* anterior es la siguiente:

```

CREATE SOURCE CONNECTOR `connect-file-pulse-csv` WITH(
  "connector.class" =
'io.streamthoughts.kafka.connect.filepulse.source.FilePulseSourceConnector',
  "task.reader.class" =
'io.streamthoughts.kafka.connect.filepulse.reader.LocalRowFileInputReader',
  "fs.cleanup.policy.class" =
'io.streamthoughts.kafka.connect.filepulse.fs.clean.LogCleanupPolicy',
  "fs.cleanup.policy.triggered.on" = 'COMMITTED',
  "fs.listing.class" =
'io.streamthoughts.kafka.connect.filepulse.fs.LocalFSDirectoryListing',
  -- El siguiente parámetro deberá corresponder con el directorio
  -- donde se almacene el fichero CSV con los datos para el ejemplo
  "fs.listing.directory.path" = '/home/appuser/FilePulse/Metal/',
  "fs.listing.filters" =
'io.streamthoughts.kafka.connect.filepulse.fs.filter.RegexFileListFilter',
  "fs.listing.interval.ms" = '10000',

```

```

    "file.filter.regex.pattern" = '.*\\.csv$',
    "offset.policy.class" =
'io.streamthoughts.kafka.connect.filepulse.offset.DefaultSourceOffsetPolicy',
    "offset.attributes.string" = 'name',
    "skip.headers" = '1',
    "topic" = 'connect-file-pulse-csv',
    "tasks.reader.class" =
'io.streamthoughts.kafka.connect.filepulse.fs.reader.LocalRowFileInputReader',
    "tasks.file.status.storage.class" =
'io.streamthoughts.kafka.connect.filepulse.state.KafkaFileObjectStateBackingStore',
    "tasks.file.status.storage.bootstrap.servers" = 'kafka:9092',
    "tasks.file.status.storage.topic" = 'connect-file-pulse-status',
    "filters" = 'ParseCSVLine,FileName,ReleaseToInt',
    "filters.ParseCSVLine.extract.column.name" = 'headers',
    "filters.ParseCSVLine.trim.column" = 'true',
    "filters.ParseCSVLine.separator" = ';',
    "filters.ParseCSVLine.type" =
'io.streamthoughts.kafka.connect.filepulse.filter.DelimitedRowFilter',
    "filters.FileName.type" =
'io.streamthoughts.kafka.connect.filepulse.filter.AppendFilter',
    "filters.FileName.field" = '$.filename',
    "filters.FileName.value" = '$metadata.name',
    "filters.ReleaseToInt.type" =
'io.streamthoughts.kafka.connect.filepulse.filter.ConvertFilter',
    "filters.ReleaseToInt.field" = 'release',
    "filters.ReleaseToInt.to" = 'INTEGER',
    "filters.ReleaseToInt.default" = '0',
    "internal.kafka.reporter.bootstrap.servers" = 'kafka:9092',
    "internal.kafka.reporter.topic" = 'connect-file-pulse-status',
    "tasks.max" = '1');

```

Para comprender qué está ocurriendo es preciso monitorizar el log de los contenedores. En particular, del servidor **ksqldb** invocando la orden ***docker logs -f ksqldb-server***

Si todo ha ido correctamente, en este momento ya se tiene lo necesario para capturar datos y almacenarlos en *Kafka*. Las órdenes siguientes permitirán el procesamiento, en este caso vía *ksql*, como si *Kafka* fuera una base de datos

En primer lugar procede conectar al servidor *ksqldb* desde el (contenedor) cliente. Las órdenes son las siguientes:

```

## Comprobaciones via cliente ksql
## Invocar cliente (en contenedor), listar tópico y ver extracto de datos
docker exec -it ksqldb-cli ksql http://ksqldb-server:8088
list topics;
print `connect-file-pulse-csv` from beginning limit 5;

```

En este punto pasamos a preparar la estructuras que soportarán la interpretación y el procesamiento de los datos. En concreto, se muestran dos alternativas según que se especifiquen metadatos de forma implícita o explícita:

```
-- Definir stream raw
-- En este caso los tipos se obtienen directamente del parser Kafka
CREATE STREAM `RAW_FILE_PULSE` WITH (KAFKA_TOPIC='connect-file-pulse-base-csv', value_format='AVRO');

-- Definir typed stream
-- Los tipos se definen explícitamente (siempre más recomendable)
CREATE STREAM `raw_songs` (
  title STRING,
  album STRING,
  duration STRING,
  artist STRING,
  type STRING,
  release INTEGER,
  filename VARCHAR)
WITH (kafka_topic='connect-file-pulse-csv', value_format='AVRO');

-- Comprobaciones. Acceso a metadatos
describe extended `RAW_FILE_PULSE`;
describe extended `raw_songs`;
```

Por último, el ejemplo concluye mostrando el streaming de los datos en *Kafka*:

```
-- Acceso a datos
SET 'auto.offset.reset'='earliest';
select * from `raw_songs` emit changes limit 10;
```